

The grammar obviously has its own namespace maintained by Mozilla. XUL is based entirely on open web standards such as CSS, JavaScript and DOM. This means that, similar to many of Mozilla's pet technologies, it's very easy to get involved in, if you're familiar with those languages. Obviously, being designed as a Mozilla specific entity, XUL has no formal specification and doesn't inter-operate with non-Gecko implementations of layout rendering. There are physical separations for the structure of add-ons that are built this way. The three segregations are:

- ▶ content
- ▶ locale
- ▶ skin

While the names are mostly self-explanatory, for the sake of detailed investigation of the platform, let's go through them sequentially. Aside from the the essential files that are needed to be contained in an add-on, an XUL-based add-on segregates these three classes of components into their namesake directories.

- ▶ **The content/ directory:** holds the browser overlay with XUL (the .xul file), along with the corresponding JavaScript file. As the name would suggest, this directory holds all the portions of the code that will constitute the end-result. In other words, any code that's meant to perform tasks other than print text or apply a style to the interface that's being provided in the adjoining XUL, goes into the JS file.
- ▶ **The locale/ directory:** holds the localisation modules based on different regions and their respective languages. This is obviously not an essential feature, but the fact that it's supported is a great boost to the credibility and outreach of one of the largest platforms for application development. It's designed to contain dumps of text segregated by languages, which can be used in the running of the code based on preferential inputs.
- ▶ **The skin/ directory:** holds the styling information for the add-on. All the CSS (Cascading Style Sheets) associated classes and files go into this directory, which can be accessed by the add-on when it's loaded.

One important question that should pop up is: how exactly does the XUL used for defining add-ons interface with the default Firefox code? Well, there are well defined "merge points" for the two sets of XUL information, so to speak. For the default action, this is how things go down: In browser.xul we can find the status bar, which looks something like this: